Gráficos por computador

OpenGL

Shaders

Tipos de datos. *Escalares*

El lenguaje de shaders GLSL soporta tanto tipos escalares como vectoriales.

Tipo	Definición
bool	Valor booleano que puede ser true o false.
float	Cantidad de 32 bits en coma flotante formateada según IEEE-754
double	Cantidad de 64 bits en coma flotante formateada según IEEE-754
int	Entero de 32 bits con signo en complemento a 2
uint	Entero de 32 bits sin signo

Especificación completa de GLSL

https://www.cs.upc.edu/~robert/teaching/idi/GLSLangSpec.4.50.pdf

Tipos de datos. *Vectores y Matrices*

Dimensión	Tipos escalares				
Escalar	bool	float	double	int	uint
Vector de 2 elementos	bvec2	vec2	dvec2	ivec2	uvec2
Vector de 3 elementos	bvec3	vec3	dvec3	ivec3	uvec3
Vector de 4 elementos	bvec4	vec4	dvec4	ivec4	uvec4
Matriz 2x2	--	mat2	dmat2	--	--
Matriz 2x3	--	mat2x3	dmat2x3	--	--
Matriz 2x4	--	mat2x4	dmat2x4	--	--
Matriz 3x2	--	mat3x2	dmat3x2	--	--
Matriz 3x3	--	mat3	dmat3	--	--
Matriz 3x4	--	mat3x4	dmat3x4	--	--
Matriz 4x2	--	mat4x2	dmat4x2	--	--
Matriz 4x3	--	mat4x3	dmat4x3	--	--
Matriz 4x4	--	mat4	dmat4	--	--

Tipos de datos. *Vectores y matrices*

Los vectores se pueden construir a partir de escalares u otros vectores, secuencias de escalares, o desde cualquier combinación de vectores y escalares:

```
vec3 foo = vec3(1.0);
vec3 bar = vec3(foo);
vec4 baz = vec4(1.0, 2.0, 3.0, 4.0);
vec4 bat = vec4(1.0, foo);
```

Código Fuente GLSL

Se puede acceder de las siguientes formas:

```
vec4 foo;
float x = foo[0];
float y = foo[1];
float z = foo[2];
float w = foo[3];
```

Código Fuente GLSL

Tipos de datos. *Vectores y matrices*

Se puede acceder a cualquier posición o posiciones de un vector por sus coordenadas y operar con ellas.

- Incluso se puede alterar el orden de acceso a las coordenadas mediante una notación llamada Swizzling.

```
vec4 firstVec;
firstVec.x + firstVec.y;

vec2 oneVec;
vec4 otherVec = oneVec.xyxx;
vec3 thirdVec = otherVec.zyy;

// a esta notación se le llama Swizzling (mezcla)

vec4 someVec;
someVec.wzyx = vec4(1.0, 2.0, 3.0, 4.0); //Invierte el orden
someVec.zx = vec2(3.0, 5.0); // Asigna a la 3ª componente de someVec un 3.0 y a la primera un 5.0
```

Código Fuente GLSL

Tipos de datos. *Vectores y matrices*

Se puede acceder a los valores que forman una matriz accediendo mediante sus índices, utilizando el operador [].

```
mat3 theMatrix;
theMatrix[1] = vec3(3.0, 3.0, 3.0); // Sets the second column to all 3.0s
theMatrix[2][0] = 16.0; // Sets the first entry of the third column to 16.0.

mat3 otherMatrix;
otherMatrix[1].yzx = vec3(3.0, 1.0, 2.0);
```

Código Fuente GLSL

Tipos de datos. *Estructuras y bucles*

También se pueden realizar estructuras de control como bucles y condicionales, de forma similar a como se haría en C++.

```
float a[10];                    // Declare an array of 10 elements
float b[a.length()];            // Declare an array of the same size
mat4 c;
float d = float(c.length());    // d is now 4
int e = c[0].length();          // e is the height of c (4)

int i;
// This loop iterates 10 times
for (i = 0; i < a.length(); i++)
{
        if(b[i] > 0)
        {
                b[i] = a[i];
        }
}
```

Código Fuente GLSL

Tipos de variables

En los shaders hay variables con diferentes propósitos en función de dónde provienen y a quién van destinados:

- **Uniform.**
  - Variables “globales” que se pasan desde la aplicación al shader.
  - Sirven para enviar datos genéricos del objeto o propiedades de la escena (posición de la luz, material del objeto, matrices para las transformaciones…).
- **Attribute**.
  - Propiedades enviadas desde la aplicación y definidas para cada vértice de manera individual. **Vertex shader: in.**
  - Sirve para pasar propiedades como la normal, el color del vértice o las coordenadas de textura.
- **Varying.**
  - Variables que se pasan del vertex shader al fragment shader y que son interpoladas de manera automática durante la rasterización. **Vertex shader: out. Fragment shader: in**.
  - Nos permite definir propiedades por vértice que luego se interpolan a los fragmentos generados durante la rasterización de la primitiva.

Funciones matemáticas

Podemos emplear un amplio repertorio de funciones matemáticas.

- **Funciones trigonométricas:**
  - float radians(float degrees)
  - vec2 radians(vec2 degrees)
  - vec3 radians(vec3 degrees)
  - vec4 radians(vec4 degrees)
  - float degrees(float radians)
  - float sin(float angle)
  - cos(), tan(), asin(), acos(), atan()
- **Funciones exponenciales:**
  - float pow(float x, float y)
  - exp(), log(), exp2(), log2(), sqrt(), inversqrt()

Funciones matemáticas

Podemos emplear un amplio repertorio de funciones matemáticas.

- **Funciones comunes:**
  - abs(), sign(), floor(), ceil(), fract(), mod(), max(), min()
  - float clamp(float x, float minVal, float maxVal)
  - mix(), step(), smoothstep()
- **Funciones geométricas:**
  - length(), distance(), dot(), cross(), normalize()
  - faceforward(), reflect(), refract()
- Consultar referencia de funciones en el manual de Khronos
  - https://www.khronos.org/files/opengl46-quick-reference-card.pdf

Tipos de shaders

Como ya vimos, los dos shaders más importantes que se suelen programar son:

- **Vertex shader.**
  - Se ejecuta una vez por vértice.
  - Aplica las transformaciones para convertirlo a coordenadas de recorte.
  - Calcula datos adicionales para pasárselos al fragment shader.
  - Variable de salida: **gl_Position**.
- **Fragment shader.**
  - Se ejecuta para los fragmentos generados durante la rasterización de la primitiva.
  - Define el color del fragmento en base a cálculos complejos:
    - Materiales.
    - Texturas.
    - Ecuación de iluminación.
  - Variable de salida: **gl_FragColor** (legacy).

Tipos de shaders

Un vertex shader se invoca una vez por vértice de entrada.

Un fragment shader se ejecuta tantas veces como fragmentos generados en la rasterización del triángulo.

- Si tenemos un objeto con un fragment shader muy complejo, hemos de tener cuidado porque puede consumir muchos recursos.

Existen otros tipos de shaders como los geometry shaders, que nos permiten crear geometría desde el propio shader, pero se escapan del alcance de la asignatura.

Como ejercicio, investiga qué otros tipos de shaders existen y para qué sirven.

Tipos de shaders

Ejemplo de Vertex shader:

```
attribute vec4 a_Position; //In: Posición del vértice
attribute vec3 a_Color; //In: Color del vértice
uniform mat4 mvp; //Matriz model view projection
varying vec3 v_Color; //Out: color para frag shader

void main()
{
	gl_Position = mvp * a_Position; //Pos del vértice en clip space
	v_Color = a_Color; //Color del vértice a ser interpolado
}
```

Código Fuente GLSL

Tipos de shaders

Ejemplo de Fragment shader:

```
varying vec3 v_Color; //In: color interpolado


void main()
{
      gl_FragColor = vec4(v_Color,1.0); // Ponemos el color del fragmento
}
```

Código Fuente GLSL

Shaders en nuestra práctica

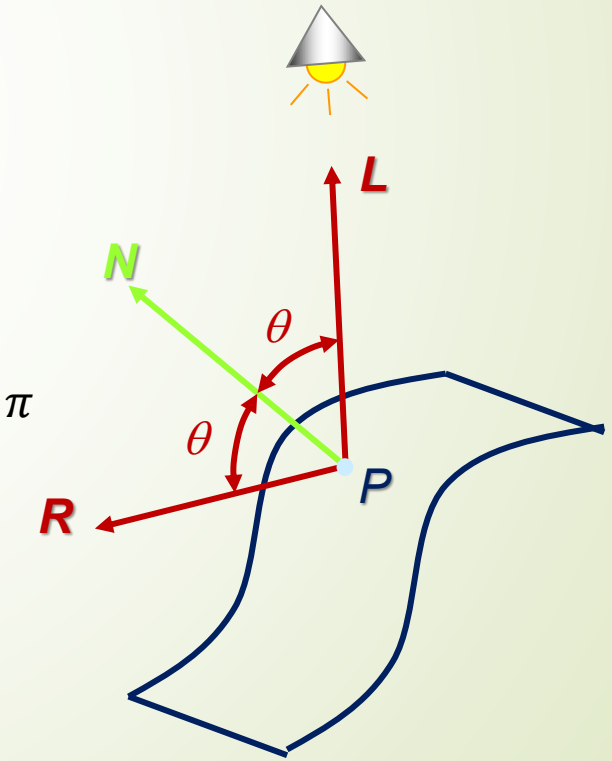
Finalmente veamos los shaders que vamos a emplear en nuestra práctica de OpenGL.

- Para ello emplearemos, de momento, sólo la iluminación ambiente y difusa.

$$I_{Phong} = I_{ambiente} + I_{difusa} + I_{especular}$$

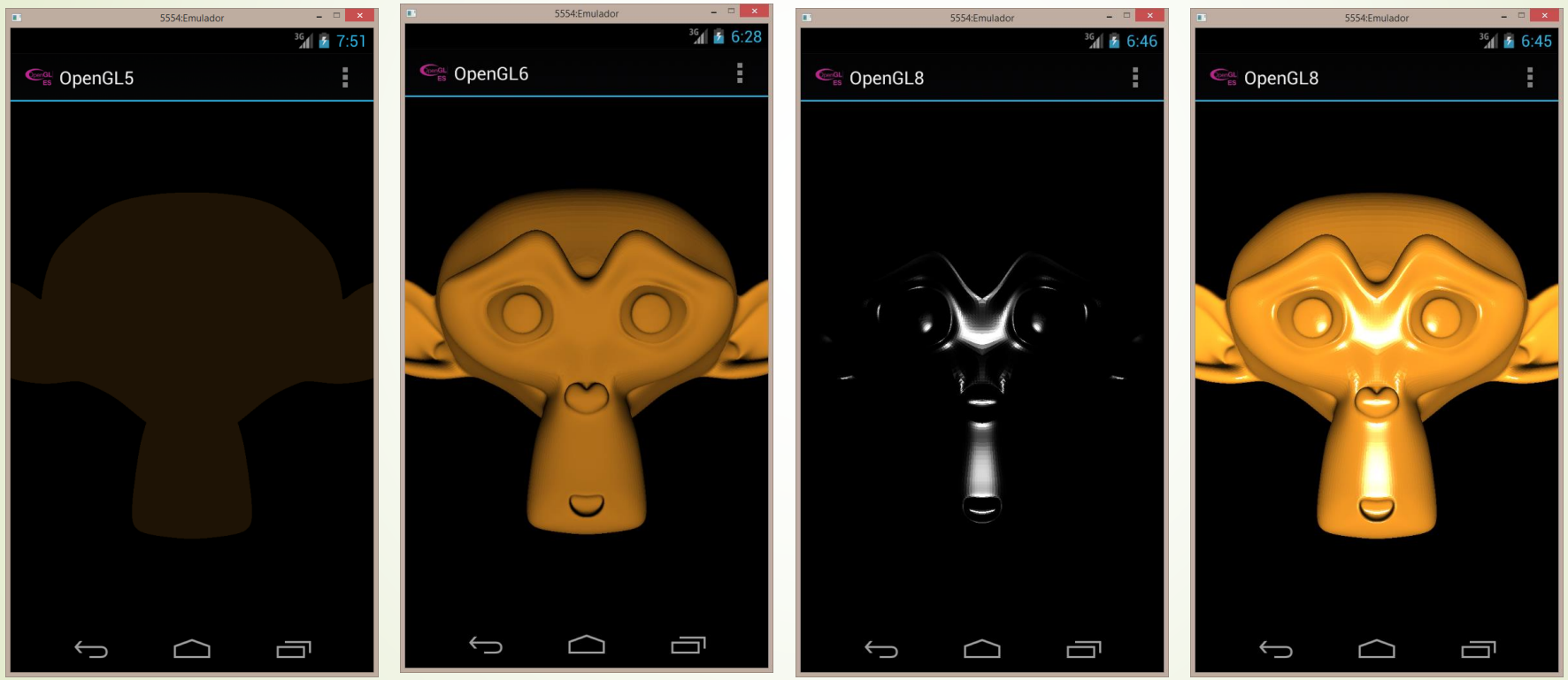
$$I_{ambiental} = I_a \, k_a$$

$$I_{difusa} = \mathbf{I_l k_d \max(\cos\theta, 0)} = I_l \, k_d \, max(L \cdot N, 0) \quad 0 \leq \theta \leq \pi$$



Shaders en nuestra práctica

Suma de los tres tipos de iluminación:



Ambiente + Difusa + Especular = Phong

Shaders en nuestra práctica

Vertex Shader (código principal):

```
void main()
{
    vec3 LightPos = vec3(0, 100, -70);        // Posición de la luz [fija]
    vec3 P = vec3(u_MVMatrix * a_Position);   // Posición del vértice
    // Normal del vértice asumiendo escala uniforme
    vec3 N = normalize(vec3(u_MVMatrix * vec4(a_Normal, 0.0)));
    float d = length(LightPos - P);           // distancia de la luz
    vec3  L = normalize(LightPos - P);        // Vector Luz

    float ambient = 0.15;
    float diffuse = max(dot(N, L), 0.0);      // Cálculo de la int. difusa
    // Cálculo de la atenuación
    float attenuation = 1.0/(0.3+(0.1*d)+(0.01*d*d));
    diffuse = diffuse*attenuation;
    float intensity = ambient + diffuse;
    v_Color = u_Color * intensity;
    mat4 MVPMatrix = u_ProjectionMatrix * u_MVMatrix;
    gl_Position = MVPMatrix * a_Position;
}
```

Código Fuente GLSL

Shaders en nuestra práctica

Fragment shader:

```
varying vec4 v_Color;       // in: color del vertex shader interpolado

void main()
{
        gl_FragColor = v_Color;
}
```

Código Fuente GLSL

Usamos componentes del modelo de reflexión de Phong, pero en este ejemplo la iluminación se calcula por vértice y posteriormente se interpola entre fragmentos.